

TEMAS SELECTOS DE CÓMPUTO DE ALTO RENDIMIENTO PARA SIMULACIONES FÍSICAS

Análisis Numérico y Computacional
Ecuación de Calor/Laplace en 1D y 2D
Dr. Juan Carlos Cajas

Autor: Emilio Ramirez

14 de mayo de 2026

Resumen

Este documento presenta la formulación matemática, discretización por diferencias finitas y evaluación computacional de solucionadores para ecuaciones de tipo calor/Laplace. Se usa el algoritmo de Thomas para sistemas tridiagonales y se analiza el rendimiento de la paralelización con OpenMP en el caso bidimensional.

1 Introducción

El objetivo es documentar una línea base reproducible para comparar variantes de implementación sin cambiar el paradigma matemático ni de paralelización.

2 Modelo matemático

2.1 Caso 1D estacionario

$$\frac{d}{dx} \left(k \frac{dT}{dx} \right) + \dot{q} = 0 \quad (1)$$

Para k constante (material isotrópico y ΔT pequeños) y sin fuentes internas ($\dot{q} = 0$), se obtiene la Ecuación de Laplace:

$$\frac{d^2 T}{dx^2} = 0 \quad (2)$$

2.2 Caso 2D estacionario

Para el caso bidimensional estacionario, se obtiene la ecuación de Laplace en 2D:

$$\frac{\partial}{\partial x} \left(k \frac{\partial T}{\partial x} \right) + \frac{\partial}{\partial y} \left(k \frac{\partial T}{\partial y} \right) + \dot{q} = 0 \quad (3)$$

$$\Rightarrow \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \quad (4)$$

$$\Rightarrow \nabla^2 T = 0 \quad (5)$$

2.3 Condiciones de frontera

Se consideran condiciones de frontera de tipo Dirichlet (temperatura fija) y Neumann (flujo de calor especificado) según el experimento.

Frontera tipo Dirichlet:

$$T(0) = T_0, \quad T(L) = T_L \quad (6)$$

Frontera tipo Neumann:

Si q_N representa el flujo de calor saliente normal a la frontera, la condición de Neumann se escribe como:

$$-k \nabla T \cdot \mathbf{n} = q_N. \quad (7)$$

En el caso unidimensional, con normal exterior $\mathbf{n} = -1$ en $x = 0$ y $\mathbf{n} = 1$ en $x = L$, esto equivale a:

$$k \frac{dT}{dx} \Big|_{x=0} = q_0, \quad -k \frac{dT}{dx} \Big|_{x=L} = q_L. \quad (8)$$

3 Discretizacion del caso 1D

La ecuación de Laplace se discretiza usando diferencias finitas.

3.1 Ecuación interior

Se parte de la ecuación diferencial Eq. (2) discretizada:

$$\frac{\Delta}{\Delta x} \left(\frac{\Delta T}{\Delta x} \right) = 0 \quad (9)$$

$$\Rightarrow \frac{\Delta}{\Delta x} \left(\frac{T(i+1) - T(i)}{\Delta x} \right) = 0 \quad (10)$$

$$\Rightarrow \frac{\Delta T(i+1) - \Delta T(i)}{(\Delta x)^2} = 0 \quad (11)$$

$$\Rightarrow \frac{(T(i+2) - T(i+1)) - (T(i+1) - T(i))}{(\Delta x)^2} = 0 \quad (12)$$

$$\Rightarrow T_{i+2} - 2T_{i+1} + T_i = 0 \quad (13)$$

Cambiando el índice $i + 1 \rightarrow i$:

$$T_{i+1} - 2T_i + T_{i-1} = 0 \quad (14)$$

con condiciones de frontera de tipo Dirichlet/Neumann según el experimento.

Ecuación matricial 1D:

La Eq. (13) se puede escribir en forma matricial $A\mathbf{T} = \mathbf{r}$, donde A es una matriz tridiagonal con -2 en la diagonal principal y 1 en las diagonales adyacentes, $\mathbf{T}$ es el vector de temperaturas desconocidas y $\mathbf{r}$ es el vector de condiciones de frontera.

$$\begin{pmatrix} 1 & -2 & 1 & 0 & 0 \cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \cdots & 0 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1 \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0 \cdots & 1 & -2 & 1 \end{pmatrix} \begin{pmatrix} T_2 \\ T_3 \\ T_4 \\ \vdots \\ T_{n-1} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \quad (15)$$

Se hace notar que faltan las temperaturas frontera i.e. T_1 y T_n que dependerán del tipo de frontera (Dirichlet o Neumann).

Ecuación matricial 1D con condiciones frontera tipo Dirichlet:

Con condiciones de frontera de tipo Dirichlet, el sistema se modifica para incluir los valores conocidos en las matrices. Dado que T_1 y T_n son conocidos:

$$T_1 = T_0, \quad T_n = T_L \quad (16)$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \cdots & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0 \cdots & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 \cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} T_0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ T_L \end{pmatrix} \quad (17)$$

Ecuación matricial 1D con frontera izquierda tipo Neumann:

Se estudia el caso en el que la frontera izquierda usa una condición de Neumann y la frontera derecha conserva una condición de Dirichlet. Con flujo saliente normal q_0 en $x = 0$:

$$k \frac{dT}{dx} \Big|_{x=0} = q_0. \quad (18)$$

Aproximando la derivada normal con una diferencia hacia adelante:

$$\frac{dT}{dx} \Big|_{x=0} \approx \frac{T_2 - T_1}{\Delta x}, \quad (19)$$

se obtiene:

$$k \frac{T_2 - T_1}{\Delta x} = q_0 \quad \Rightarrow \quad -T_1 + T_2 = \frac{q_0}{k} \Delta x \quad (20)$$

Por tanto, el sistema algebraico para una frontera izquierda Neumann y una frontera derecha Dirichlet $T_n = T_L$ queda:

$$\begin{pmatrix} -1 & 1 & 0 & 0 & 0 \cdots & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0 \cdots & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 \cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} \frac{q_0}{k} \Delta x \\ 0 \\ 0 \\ \vdots \\ 0 \\ T_L \end{pmatrix}. \quad (21)$$

Si se imponen condiciones de Neumann en ambos extremos para la ecuación de Laplace sin fuentes, el sistema queda determinado solo hasta una constante aditiva y requiere una condición adicional de referencia. Además, los flujos deben cumplir la condición de compatibilidad $q_0 + q_L = 0$.

4 Algoritmo de Thomas

Tenemos el sistema tridiagonal $A\mathbf{T} = \mathbf{r}$, donde A es una matriz tridiagonal con elementos a_i en la subdiagonal, b_i en la diagonal, y c_i en la superdiagonal. De la misma manera, $\mathbf{T}$ es el vector de temperaturas desconocidas y $\mathbf{r}$ es el vector de condiciones de frontera, con elementos d_i .

$$\begin{aligned} b_1 T_1 + c_1 T_2 &= r_1 \\ a_2 T_1 + b_2 T_2 + c_2 T_3 &= r_2 \\ a_3 T_2 + b_3 T_3 + c_3 T_4 &= r_3 \\ &\vdots \\ a_{n-1} T_{n-2} + b_{n-1} T_{n-1} + c_{n-1} T_n &= r_{n-1} \\ a_n T_{n-1} + b_n T_n &= r_n \end{aligned}$$

$$\begin{pmatrix} b_1 & c_1 & 0 & 0 & 0 \cdots & 0 & 0 & 0 \\ a_2 & b_2 & c_2 & 0 & 0 \cdots & 0 & 0 & 0 \\ 0 & a_3 & b_3 & c_3 & 0 \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0 \cdots & a_{n-1} & b_{n-1} & c_{n-1} \\ 0 & 0 & 0 & 0 & 0 \cdots & 0 & a_n & b_n \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_{n-1} \\ r_n \end{pmatrix} \quad (22)$$

Así, se tienen las ecuaciones del sistema tridiagonal a resolver:

$$a_i T_{i-1} + b_i T_i + c_i T_{i+1} = r_i, \quad i = 1, 2, \dots, n, \quad a_1 = 0, \quad c_n = 0. \quad (23)$$

Resolviendo,

$$T_1 = \frac{r_1 - c_1 T_2}{b_1} \quad (24)$$

Y definimos,

$$\tilde{r}_1 = \frac{r_1}{b_1}, \quad \tilde{c}_1 = \frac{c_1}{b_1} \quad (25)$$

Con lo cual, la primera ecuación se reescribe como:

$$T_1 = \tilde{r}_1 - \tilde{c}_1 T_2 \quad (26)$$

Si ahora resolvemos el segundo renglón para T_2 :

$$T_2 = \frac{r_2 - a_2 T_1 - c_2 T_3}{b_2} \quad (27)$$

$$\Rightarrow T_2 = \frac{r_2 - a_2(\tilde{r}_1 - \tilde{c}_1 T_2) - c_2 T_3}{b_2} \quad (28)$$

$$\Rightarrow T_2 \left(1 - \frac{a_2 \tilde{c}_1}{b_2} \right) = \frac{r_2 - a_2 \tilde{r}_1 - c_2 T_3}{b_2} \quad (29)$$

$$\Rightarrow T_2 = \frac{r_2 - a_2 \tilde{r}_1 - c_2 T_3}{b_2 - a_2 \tilde{c}_1} \quad (30)$$

Y si ahora definimos

$$\tilde{r}_2 = \frac{r_2 - a_2 \tilde{r}_1}{b_2 - a_2 \tilde{c}_1}, \quad \tilde{c}_2 = \frac{c_2}{b_2 - a_2 \tilde{c}_1} \quad (31)$$

Entonces nos queda,

$$T_2 = \tilde{r}_2 - \tilde{c}_2 T_3 \quad (32)$$

En general, para $i = 2, 3, \dots, n-1$ se tiene:

$$T_i = \tilde{r}_i - \tilde{c}_i T_{i+1}, \quad (33)$$

con:

$$\tilde{r}_i = \frac{r_i - a_i \tilde{r}_{i-1}}{b_i - a_i \tilde{c}_{i-1}}, \quad \tilde{c}_i = \frac{c_i}{b_i - a_i \tilde{c}_{i-1}} \quad (34)$$

Para T_n se tiene:

$$T_n = \frac{r_n - a_n T_{n-1}}{b_n} \quad (35)$$

$$\Rightarrow T_n = \frac{r_n - a_n (\tilde{r}_{n-1} - \tilde{c}_{n-1} T_n)}{b_n} \quad (36)$$

$$\Rightarrow T_n \left(1 - \frac{a_n \tilde{c}_{n-1}}{b_n} \right) = \frac{r_n - a_n \tilde{r}_{n-1}}{b_n} \quad (37)$$

$$\Rightarrow T_n = \frac{r_n - a_n \tilde{r}_{n-1}}{b_n - a_n \tilde{c}_{n-1}} \quad (38)$$

$$\Rightarrow T_n = \tilde{r}_n \quad (39)$$

Conociendo T_n , se obtiene T_{n-1} , luego T_{n-2} , y así sucesivamente hasta T_1 .

Como ejemplo, se muestra un pequeño caso de una frontera izquierda Neumann con flujo q_0 y una frontera derecha Dirichlet con temperatura $T_L = 0$. Para $n = 5$ y una longitud $L = 10$ m, los coeficientes de la matriz tridiagonal y el vector de resultados son:

$$A = \begin{pmatrix} -1 & 1 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0 \\ 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{r} = \begin{pmatrix} \frac{q_0}{k} \Delta x \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad (40)$$

Tenemos entonces, estableciendo que $q_0/k = -1.0$ K/m y $\Delta x = \frac{L}{n-1} = \frac{10 \text{ m}}{5-1} = 2.5$ m:

$$\begin{array}{llll} a_1 = 0 & b_1 = -1 & c_1 = 1 & r_1 = \frac{q_0}{k} \Delta x = -2.5 \\ a_2 = 1 & b_2 = -2 & c_2 = 1 & r_2 = 0 \\ a_3 = 1 & b_3 = -2 & c_3 = 1 & r_3 = 0 \\ a_4 = 1 & b_4 = -2 & c_4 = 1 & r_4 = 0 \\ a_5 = 0 & b_5 = 1 & c_5 = 0 & r_5 = 0 \end{array} \quad (41)$$

Con estos valores podemos calcular los coeficientes modificados $\tilde{c}_i$ y el lado derecho modificado $\tilde{r}_i$ para el algoritmo de Thomas:

$$\begin{array}{ll} \tilde{c}_1 = \frac{c_1}{b_1} = -1 & \tilde{r}_1 = \frac{r_1}{b_1} = 2.5 \\ \tilde{c}_2 = \frac{c_2}{b_2 - a_2 \tilde{c}_1} = \frac{1}{-2 - 1(-1)} = -1 & \tilde{r}_2 = \frac{r_2 - a_2 \tilde{r}_1}{b_2 - a_2 \tilde{c}_1} = \frac{0 - 1(2.5)}{-2 - 1(-1)} = 2.5 \\ \tilde{c}_3 = \frac{c_3}{b_3 - a_3 \tilde{c}_2} = \frac{1}{-2 - 1(-1)} = -1 & \tilde{r}_3 = \frac{r_3 - a_3 \tilde{r}_2}{b_3 - a_3 \tilde{c}_2} = \frac{0 - 1(2.5)}{-2 - 1(-1)} = 2.5 \\ \tilde{c}_4 = \frac{c_4}{b_4 - a_4 \tilde{c}_3} = \frac{1}{-2 - 1(-1)} = -1 & \tilde{r}_4 = \frac{r_4 - a_4 \tilde{r}_3}{b_4 - a_4 \tilde{c}_3} = \frac{0 - 1(2.5)}{-2 - 1(-1)} = 2.5 \\ \tilde{c}_5 = \frac{c_5}{b_5 - a_5 \tilde{c}_4} = 0 & \tilde{r}_5 = \frac{r_5 - a_5 \tilde{r}_4}{b_5 - a_5 \tilde{c}_4} = 0 \end{array} \quad (42)$$

Y ahora podemos resolver hacia atrás para obtener las temperaturas:

$$\begin{array}{l} T_5 = \tilde{r}_5 = 0 \\ T_4 = \tilde{r}_4 - \tilde{c}_4 T_5 = 2.5 - (-1)(0) = 2.5 \\ T_3 = \tilde{r}_3 - \tilde{c}_3 T_4 = 2.5 - (-1)(2.5) = 5 \\ T_2 = \tilde{r}_2 - \tilde{c}_2 T_3 = 2.5 - (-1)(5) = 7.5 \\ T_1 = \tilde{r}_1 - \tilde{c}_1 T_2 = 2.5 - (-1)(7.5) = 10 \end{array} \quad (43)$$

Por lo tanto, la solución del sistema es:

$$\mathbf{T} = \begin{pmatrix} 10 \\ 7.5 \\ 5 \\ 2.5 \\ 0 \end{pmatrix} \quad (44)$$

A continuación se muestra la implementación en Fortran del algoritmo de Thomas para resolver el sistema tridiagonal resultante de la discretización de la ecuación de Laplace en 1D.

5 Metodología computacional del caso 1D

Para el caso unidimensional se resuelve directamente el sistema tridiagonal producido por la discretización de Laplace:

$$A\mathbf{T} = \mathbf{r}. \quad (45)$$

Se implementa el algoritmo de Thomas para resolver este sistema de forma eficiente. Se usa el comando `gfortran tridiagonal.f90 calor1D.f90 -o calor1D` para compilar la versión secuencial. El programa se ejecuta con `./calor1D` y los resultados se guardan en archivos de texto para su posterior análisis.

Se muestra a continuación el código Fortran de la versión secuencial para el caso 1D. Se comienza abriendo el programa y declarando las variables necesarias para el algoritmo:

```
Program Calor1D
!
Implicit none
!
! Declaracion de variables
!
! Iteradores y tamaño del problema
!
integer :: ii
integer, parameter :: L = 10 ! metros
integer, parameter :: nn = 60
double precision, parameter :: q0_sobre_k = -1.d0
!
! Variables del dominio computacional
!
double precision :: deltax
!
! Variables del problema f\'isico
!
! double precision :: aa(nn,nn) ! matriz para el problema algebraico
double precision :: tt(nn)      ! vector de inc\'ognitas
double precision :: rr(nn)      ! vector de resultados del s. ecuaciones
!
double precision :: aa(nn), bb(nn), cc(nn) ! Variables para almacenar
                                         ! las diagonales de la matriz tridiagonal
double precision :: cp(nn), den(nn)
```

Establecemos el dominio computacional y el paso de malla $\Delta x = L/(nn - 1)$: El valor $L = 10$ metros define la longitud total del dominio de solución; se elige arbitrariamente para este problema de prueba. La discretización se realiza con $nn = 60$ nodos (59 dominios), resultando en $\Delta x = L/(nn - 1) = 10/59$

```
! Dominio computacional
deltax = dble(L)/(nn-1)
```

Luego, se inicializan los vectores necesarios para el algoritmo de Thomas. Se hace notar que no se construye la matriz completa A , sino que se almacenan solamente las diagonales a , b y c en los vectores correspondientes.

```
! Inicializacion de variables
!
aa(:) = 0.d0
bb(:) = 0.d0
cc(:) = 0.d0
rr(:) = 0.d0
```

Se ensamblan los coeficientes de la matriz tridiagonal y el vector de resultados según la discretización de la ecuación de Laplace. Para los nodos interiores, los coeficientes son constantes: $a_i = 1$, $b_i = -2$ y $c_i = 1$.

```
! Ensamblar la matriz tridiagonal y el vector de resultados
do ii = 2, nn-1
  !
  aa(ii) = 1.d0
  bb(ii) = -2.d0
  cc(ii) = 1.d0
  rr(ii) = 0.d0
  !
end do
```

A continuación, se imponen las condiciones de frontera. En este caso, se asume una condición de Neumann en el extremo izquierdo ($x = 0$) con un flujo tal que $q_0/k = -1.0$ K/m, y una condición de Dirichlet en el extremo derecho ($x = L$) con temperatura $T_L = 0$. Esto se traduce en los coeficientes de la primera y última ecuaciones del sistema.

```
! Impone cond. frontera
!
aa(1)      = 0.d0 ! no se usa en los c'alculos
bb(1)      = -1.d0
cc(1)      = 1.d0
rr(1)      = q0_sobre_k * deltax
!
aa(nn)     = 0.d0
bb(nn)     = 1.d0
cc(nn)     = 0.d0 ! no se usa en los c'alculos
rr(nn)     = 0.d0
```

Se resuelve el sistema tridiagonal usando el algoritmo de Thomas, que se implementa en las subrutinas `tri_factor` y `tri_solve`. Estas subrutinas realizan la factorización y la resolución del sistema de forma eficiente, aprovechando la estructura tridiagonal de la matriz.

```
! Resolver el problema algebraico
!
call tri_factor(aa,bb,cc,cp,den,nn)
call tri_solve(aa,cp,den,rr,tt,nn)
```

Finalmente, se guarda el resultado en un archivo de texto para su posterior análisis. Se escribe el vector de temperaturas `tt` junto con las posiciones correspondientes en el dominio.

```
! Postproceso
!
do ii = 1, nn
  write(101,*) (ii-1)*deltax, tt(ii)
end do
!
end Program Calor1D
```

Subrutina tri

La subrutina `tri` es adicional y no se usa para la resolución del sistema tridiagonal, pero se incluye para ilustrar el proceso de eliminación gaussiana y sustitución hacia atrás.

```
subroutine tri(a,b,c,r,u,nn)
  !
  ! Subrutina que resuelve el problema usando eliminaci\'on gaussiana
```

```

!
! Ax=r donde A es una matriz tridiagonal con elementos a, b, c
! y devuelve el valor de u
!
implicit none
!
integer,          intent(in)      :: nn
double precision, intent(in)      :: a(nn),c(nn)
double precision, intent(inout)   :: b(nn),r(nn)
double precision, intent(inout)   :: u(nn)
integer :: ii
double precision :: m

```

Para el barrido hacia adelante se usa (*gausselim*): Para $i = 2, 3, \dots, n$, se calcula el multiplicador de eliminación y se actualizan la diagonal y el lado derecho *in-place*:

$$m_i = \frac{a_i}{b_{i-1}}, \quad (46)$$

$$b_i^* \leftarrow b_i - m_i c_{i-1}, \quad (47)$$

$$r_i^* \leftarrow r_i - m_i r_{i-1}. \quad (48)$$

```

! Eliminaci\'on elementos bajo la matriz
gausselim: do ii=2,nn
  m = a(ii)/b(ii-1)
  r(ii)=r(ii)-m*r(ii-1)
  b(ii)=b(ii)-m*c(ii-1)
end do gausselim
!

```

Tras este barrido el sistema original $A\mathbf{u} = \mathbf{r}$ ha quedado reducido a uno triangular superior equivalente $U\mathbf{u} = \mathbf{r}^*$, donde $\mathbf{r}^*$ denota el lado derecho transformado.

Sustitución hacia atrás (*sustatras*): Se obtiene la última incógnita directamente y luego se retrocede:

$$u_n = \frac{r_n^*}{b_n^*}, \quad (49)$$

$$u_i = \frac{r_i^* - c_i u_{i+1}}{b_i^*}, \quad i = n-1, n-2, \dots, 1. \quad (50)$$

```

! Soluci\'on para u
!
u(nn)=r(nn)/b(nn)
sustatras: do ii=nn-1,1,-1
  u(ii)=(r(ii)-c(ii)*u(ii+1))/b(ii)
end do sustatras
!
end subroutine tri

```

El coste total también es $\mathcal{O}(n)$ operaciones, frente a los $\mathcal{O}(n^3)$ de una eliminación gaussiana general. Se declara esta subrutina para ilustrar el proceso completo, aunque en la implementación final se opta por usar las subrutinas `tri_factor` y `tri_solve` que realizan la factorización y la resolución de forma separada.

Subrutina `tri_factor`:

Se explica a continuación el código de la subrutina `tri_factor`, que realiza la factorización de Thomas para una matriz tridiagonal constante.

Se declaran las variables de entrada y salida, incluyendo los vectores de coeficientes de la matriz tridiagonal y los vectores para almacenar la factorización.

```

subroutine tri_factor(a,b,c,cp,den,nn)
!
! Factorizacion de Thomas para una matriz tridiagonal constante.
!
implicit none
!
integer,          intent(in)  :: nn
double precision, intent(in)  :: a(nn),b(nn),c(nn)
double precision, intent(out) :: cp(nn),den(nn)
integer :: ii

```

Se inicializan los vectores de la factorización según las ecuaciones 25 y 31:

```

den(1) = b(1)
cp(1)  = c(1)/den(1) ! cp \equiv c'

do ii=2,nn-1
    den(ii) = b(ii) - a(ii)*cp(ii-1)
    cp(ii)  = c(ii)/den(ii)
end do
den(nn) = b(nn) - a(nn)*cp(nn-1)
cp(nn)  = 0.d0
end subroutine tri_factor

```

Subrutina tri_solve:

Se muestra a continuación el código de la subrutina `tri_solve`, que realiza la sustitución hacia atrás para resolver el sistema tridiagonal usando la factorización calculada por `tri_factor`.

```

subroutine tri_solve(a,cp,den,r,u,nn)
!
! Sustituci\'on usando la factorizacion calculada por tri_factor.
! Modifica r in-place para guardar el lado derecho transformado.
!
implicit none
!
integer,          intent(in)  :: nn
double precision, intent(in)  :: a(nn),cp(nn),den(nn)
double precision, intent(inout) :: r(nn)
double precision, intent(inout) :: u(nn)
integer :: ii

```

Se realiza el barrido hacia adelante según la ecuación 34 para transformar el lado derecho usando la factorización calculada.

```

r(1) = r(1)/den(1)
do ii=2,nn
    r(ii) = (r(ii) - a(ii)*r(ii-1))/den(ii)
end do

```

Finalmente, usando las ecuaciones 35 y 39, se obtiene el vector de temperaturas u mediante sustitución hacia atrás.

```

u(nn) = r(nn)
do ii=nn-1,1,-1
    u(ii) = r(ii) - cp(ii)*u(ii+1)
end do
end subroutine tri_solve

```

Resultados

Se muestran a continuación los resultados obtenidos para el caso 1D, incluyendo la distribución de temperatura a lo largo del dominio de solución. Se observa que la temperatura disminuye linealmente desde el extremo izquierdo, donde se impone un flujo de calor, hasta el extremo derecho, donde se fija la temperatura a cero.

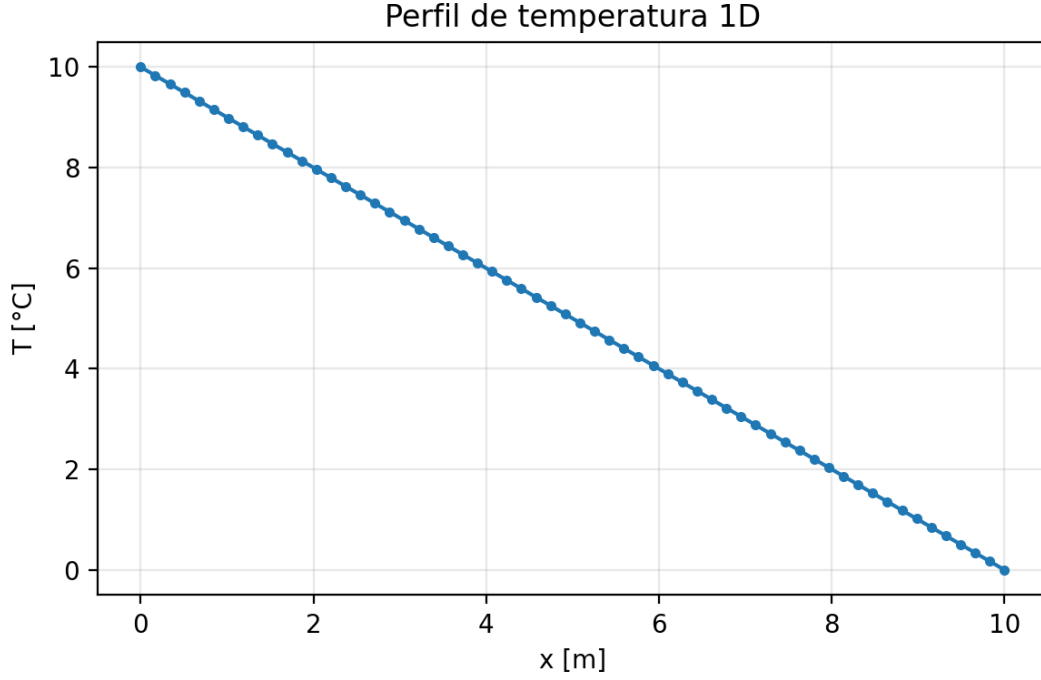


Figura 1: Distribución de temperatura en el caso 1D

6 Discretización del caso 2D

En el caso bidimensional, la ecuación de Laplace se discretiza como:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta x)^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta y)^2} = 0. \quad (51)$$

Separando los términos en la dirección x :

$$\frac{1}{(\Delta x)^2} T_{i-1,j} - 2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right) T_{i,j} + \frac{1}{(\Delta x)^2} T_{i+1,j} = -\frac{T_{i,j-1} + T_{i,j+1}}{(\Delta y)^2}. \quad (52)$$

Esta expresión genera un sistema tridiagonal por cada línea horizontal j :

$$a_i^{(x)} T_{i-1,j} + b_i^{(x)} T_{i,j} + c_i^{(x)} T_{i+1,j} = r_i^{(x)}. \quad (53)$$

Los coeficientes usados son:

$$a_i^{(x)} = \frac{1}{(\Delta x)^2}, \quad (54)$$

$$b_i^{(x)} = -2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right), \quad (55)$$

$$c_i^{(x)} = \frac{1}{(\Delta x)^2}, \quad (56)$$

$$r_i^{(x)} = -\frac{T_{i,j-1} + T_{i,j+1}}{(\Delta y)^2}. \quad (57)$$

De forma análoga, separando los términos en la dirección y :

$$\frac{1}{(\Delta y)^2} T_{i,j-1} - 2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right) T_{i,j} + \frac{1}{(\Delta y)^2} T_{i,j+1} = - \frac{T_{i-1,j} + T_{i+1,j}}{(\Delta x)^2}. \quad (58)$$

Esto produce sistemas tridiagonales verticales:

$$a_j^{(y)} T_{i,j-1} + b_j^{(y)} T_{i,j} + c_j^{(y)} T_{i,j+1} = r_j^{(y)}, \quad (59)$$

con:

$$a_j^{(y)} = \frac{1}{(\Delta y)^2}, \quad (60)$$

$$b_j^{(y)} = -2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right), \quad (61)$$

$$c_j^{(y)} = \frac{1}{(\Delta y)^2}, \quad (62)$$

$$r_j^{(y)} = - \frac{T_{i-1,j} + T_{i+1,j}}{(\Delta x)^2}. \quad (63)$$

El proceso iterativo se detiene cuando el cambio entre dos iteraciones consecutivas es menor que una tolerancia prescrita. El residuo usado es:

$$R^{(k)} = \left[\sum_{i=1}^{n_x} \sum_{j=1}^{n_y} \left(T_{i,j}^{(k)} - T_{i,j}^{(k-1)} \right)^2 \right]^{1/2}. \quad (64)$$

El criterio de convergencia es:

$$R^{(k)} < \varepsilon. \quad (65)$$

Se usa como ejemplo una placa bidimensional con dimensiones $L_x = 8$ m y $L_y = 4$ m, discretizada con $n_x = 5$ nodos en la dirección x y $n_y = 3$ nodos en la dirección y . Se imponen condiciones de frontera de Dirichlet en los extremos izquierdo y derecho con temperaturas $T_0 = 0^\circ\text{C}$ y $T_{L_x} = 10^\circ\text{C}$, mientras que en los extremos superior e inferior se imponen condiciones de Neumann con flujo de calor $q_0/k = 0$ K/m.

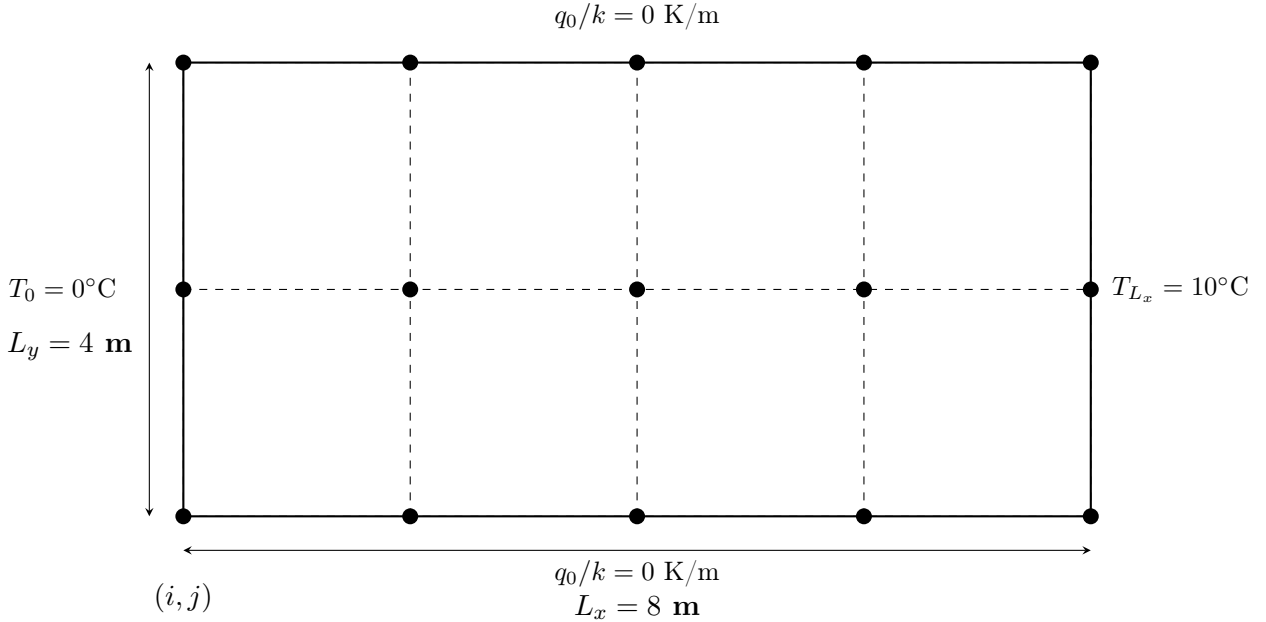


Figura 2: Placa bidimensional discretizada con $n_x = 5$ nodos en dirección x y $n_y = 3$ nodos en dirección y . Condiciones de frontera: Dirichlet en los extremos izquierdo y derecho, Neumann en los extremos superior e inferior.

Se construyen las matrices tridiagonales para cada cada dirección, una matriz para cada conjunto de nodos. Así en total, se resuelven n_y sistemas tridiagonales en la dirección x y n_x sistemas tridiagonales en la dirección y . En este caso, $3 + 5 = 8$ sistemas tridiagonales en total.

Dado que los coeficientes de los sistemas tridiagonales se mantendrán constantes en cada iteración, pues no dependen de las tmeperaturas, se pueden pre-calcular los coeficientes modificados para el algoritmo de Thomas antes de iniciar las iteraciones. Esto optimiza el proceso de resolución de los sistemas tridiagonales en cada iteración, ya que se evita recalcular los coeficientes en cada paso. Teniendo en cuenta que $\Delta x = L_x/(n_x - 1) = 2$ y $\Delta y = L_y/(n_y - 1) = 2$, los coeficientes de los sistemas tridiagonales para cada dirección se establecen como sigue:

$$\begin{aligned} a_1^{(x)} &= 0, & b_1^{(x)} &= 1, & c_1^{(x)} &= 0, \\ a_i^{(x)} &= \frac{1}{(\Delta x)^2} = 0.25, & b_i^{(x)} &= -2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right) = -1, & c_i^{(x)} &= \frac{1}{(\Delta x)^2} = 0.25, & i \in \{2, 3, 4\} \\ a_5^{(x)} &= 0, & b_5^{(x)} &= 1, & c_5^{(x)} &= 0 \end{aligned} \quad (66)$$

$$\begin{aligned} a_1^{(y)} &= 0, & b_1^{(y)} &= -1, & c_1^{(y)} &= 1, \\ a_j^{(y)} &= \frac{1}{(\Delta y)^2} = 0.25, & b_j^{(y)} &= -2 \left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2} \right) = -1, & c_j^{(y)} &= \frac{1}{(\Delta y)^2} = 0.25, \\ a_n^{(y)} &= -1, & b_n^{(y)} &= 1, & c_n^{(y)} &= 0 \end{aligned} \quad (67)$$

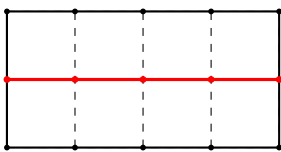
A continuación se realiza la factorización de Thomas para cada eje:

$$\begin{aligned} \tilde{c}_1^{(x)} &= \frac{c_1^{(x)}}{b_1^{(x)}} = 0, & \tilde{c}_1^{(y)} &= \frac{c_1^{(y)}}{b_1^{(y)}} = -1, \\ \tilde{c}_2^{(x)} &= \frac{c_2^{(x)}}{b_2^{(x)} - a_2^{(x)} \tilde{c}_1^{(x)}} = \frac{0.25}{-1 - 0.25 * 0} = -1/4, & \tilde{c}_2^{(y)} &= \frac{c_2^{(y)}}{b_2^{(y)} - a_2^{(y)} \tilde{c}_1^{(y)}} = \frac{0.25}{-1 - 0.25 * (-1)} = -1/3, \\ \tilde{c}_3^{(x)} &= \frac{c_3^{(x)}}{b_3^{(x)} - a_3^{(x)} \tilde{c}_2^{(x)}} = \frac{0.25}{-1 - 0.25 * (-0.25)} = -4/15, & \tilde{c}_3^{(y)} &= 0, \\ \tilde{c}_4^{(x)} &= \frac{c_4^{(x)}}{b_4^{(x)} - a_4^{(x)} \tilde{c}_3^{(x)}} = \frac{0.25}{-1 - 0.25 * (-4/15)} = -15/56, \\ \tilde{c}_5^{(x)} &= 0 \end{aligned} \quad (68)$$

Seguidamente, se imponen las condiciones de frontera:

$$\text{cfx} = \begin{pmatrix} 0 & 10 \\ 0 & 10 \\ 0 & 10 \end{pmatrix}_{3 \times 2} \quad \text{cfy} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ \vdots & \vdots \\ 0 & 0 \end{pmatrix}_{5 \times 2} \quad (69)$$

Con las condiciones iniciales establecidas, se resuelven los sistemas tridiagonales iterativamente. Se comienza con un barrido a lo largo del eje y y luego a lo largo del eje x , actualizando las temperaturas en cada iteración. En este paso se está resolviendo la segunda horizontal de la malla ($j = 2$) durante el barrido en dirección x .



Segunda horizontal activa $i = 2$

$$\begin{aligned} r_1^{(x)} &= \text{cfx}(2, 1) = 0, \\ r_2^{(x)} &= -\frac{1}{(\Delta y)^2} (T(2, 1) + T(2, 3)) = -\frac{(0 + 0)}{(\Delta y)^2} = 0, \\ r_3^{(x)} &= -\frac{1}{(\Delta y)^2} (T(3, 1) + T(3, 3)) = 0, \\ r_4^{(x)} &= -\frac{1}{(\Delta y)^2} (T(4, 1) + T(4, 3)) = 0, \\ r_5^{(x)} &= \text{cfx}(2, 2) = 10 \end{aligned} \quad (70)$$

Para este ejemplo reducido, en la primera iteración se parte de $T_{i,j}^{(0)} = 0$, con $\Delta x = \Delta y = 2$ y por tanto $1/(\Delta x)^2 = 1/(\Delta y)^2 = 0.25$. Como $\text{cfx}(2, 1) = 0$ y $\text{cfx}(2, 2) = 10$, el sistema en x para $j = 2$ queda con:

$$\mathbf{r}^{(x)} = [0, 0, 0, 0, 10]^T. \quad (71)$$

Se resuelve con Thomas en forma explícita. Para la dirección x , los coeficientes son:

$$\begin{aligned} a^{(x)} &= [0, 0.25, 0.25, 0.25, 0], \\ b^{(x)} &= [1, -1, -1, -1, 1], \\ c^{(x)} &= [0, 0.25, 0.25, 0.25, 0]. \end{aligned} \quad (72)$$

La factorización (ya calculada antes) produce:

$$\begin{aligned} \tilde{c}_1^{(x)} &= 0, \quad \tilde{c}_2^{(x)} = -\frac{1}{4}, \quad \tilde{c}_3^{(x)} = -\frac{4}{15}, \quad \tilde{c}_4^{(x)} = -\frac{15}{56}, \quad \tilde{c}_5^{(x)} = 0, \\ d_1^{(x)} &= 1, \quad d_2^{(x)} = -1, \quad d_3^{(x)} = -\frac{15}{16}, \quad d_4^{(x)} = -\frac{14}{15}, \quad d_5^{(x)} = 1. \end{aligned} \quad (73)$$

En la sustitución hacia adelante sobre $\mathbf{r}^{(x)}$:

$$\begin{aligned} \hat{r}_1 &= \frac{r_1}{d_1} = 0, \\ \hat{r}_2 &= \frac{r_2 - a_2 \hat{r}_1}{d_2} = \frac{0 - 0.25 \cdot 0}{-1} = 0, \\ \hat{r}_3 &= \frac{r_3 - a_3 \hat{r}_2}{d_3} = \frac{0 - 0.25 \cdot 0}{-15/16} = 0, \\ \hat{r}_4 &= \frac{r_4 - a_4 \hat{r}_3}{d_4} = \frac{0 - 0.25 \cdot 0}{-14/15} = 0, \\ \hat{r}_5 &= \frac{r_5 - a_5 \hat{r}_4}{d_5} = \frac{10 - 0 \cdot 0}{1} = 10. \end{aligned} \quad (74)$$

Y en la retro-sustitución:

$$\begin{aligned} t_5^{(x)} &= \hat{r}_5 = 10, \\ t_4^{(x)} &= \hat{r}_4 - \tilde{c}_4^{(x)} t_5^{(x)} = 0 - \left(-\frac{15}{56}\right) 10 = \frac{75}{28}, \\ t_3^{(x)} &= \hat{r}_3 - \tilde{c}_3^{(x)} t_4^{(x)} = 0 - \left(-\frac{4}{15}\right) \frac{75}{28} = \frac{5}{7}, \\ t_2^{(x)} &= \hat{r}_2 - \tilde{c}_2^{(x)} t_3^{(x)} = 0 - \left(-\frac{1}{4}\right) \frac{5}{7} = \frac{5}{28}, \\ t_1^{(x)} &= \hat{r}_1 - \tilde{c}_1^{(x)} t_2^{(x)} = 0. \end{aligned} \quad (75)$$

Por tanto:

$$\mathbf{t}^{(x)} = [0, 5/28, 5/7, 75/28, 10]^T \approx [0, 0.1786, 0.7143, 2.6786, 10]^T. \quad (76)$$

Estos valores actualizan la fila $j = 2$: $T(i, 2) \leftarrow t_i^{(x)}$.

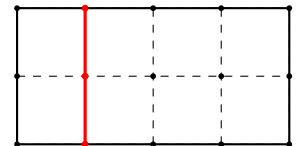
Como $n_y = 3$, y los valores con $j = 1$ y $j = 3$ pertenecen a las condiciones frontera, entonces el barrido en y para cada eje x termina acá.

Así, ahora se comienza con el barrido en x para cada eje y . Como $n_y = 3$, cada sistema vertical tiene **tres** componentes en $\mathbf{r}^{(y)}$: $r_1^{(y)}, r_2^{(y)}, r_3^{(y)}$. Para la segunda columna de la malla ($i = 2$), se tiene:

$$\begin{aligned} r_1^{(y)}(i = 2) &= \text{cfy}(2, 1) = 0, \\ r_2^{(y)}(i = 2) &= -0.25 [T(1, 2) + T(3, 2)] = -5/28, \\ r_3^{(y)}(i = 2) &= \text{cfy}(2, 2) = 0. \end{aligned} \quad (77)$$

Por tanto, para $i = 2$:

$$\mathbf{r}^{(y)}(i = 2) = [0, -5/28, 0]^T. \quad (78)$$



Segunda columna activa $i = 2$

Análogamente, para las otras columnas interiores:

$$\begin{aligned}\mathbf{r}^{(y)}(i=3) &= [0, -5/7, 0]^T, \\ \mathbf{r}^{(y)}(i=4) &= [0, -75/28, 0]^T.\end{aligned}\tag{79}$$

Para cada $i \in \{2, 3, 4\}$, el sistema vertical de tamaño 3 usa:

$$\begin{aligned}a^{(y)} &= [0, 0.25, -1], \\ b^{(y)} &= [-1, -1, 1], \\ c^{(y)} &= [1, 0.25, 0],\end{aligned}\tag{80}$$

y su factorización:

$$\tilde{c}_1^{(y)} = -1, \quad \tilde{c}_2^{(y)} = -\frac{1}{3}, \quad \tilde{c}_3^{(y)} = 0, \quad d_1^{(y)} = -1, \quad d_2^{(y)} = -\frac{3}{4}, \quad d_3^{(y)} = \frac{2}{3}.\tag{81}$$

Tomando primero $i = 2$, con $\mathbf{r}^{(y)} = [0, -5/28, 0]^T$:

$$\begin{aligned}\hat{r}_1 &= 0/(-1) = 0, \\ \hat{r}_2 &= \frac{-5/28 - 0.25 \cdot 0}{-3/4} = \frac{5}{21}, \\ \hat{r}_3 &= \frac{0 - (-1)\hat{r}_2}{2/3} = \frac{3}{2}\hat{r}_2 = \frac{5}{14}, \\ t_3^{(y)} &= \hat{r}_3 = \frac{5}{14}, \\ t_2^{(y)} &= \hat{r}_2 - \tilde{c}_2^{(y)}t_3^{(y)} = \frac{5}{21} - \left(-\frac{1}{3}\right)\frac{5}{14} = \frac{5}{14}, \\ t_1^{(y)} &= \hat{r}_1 - \tilde{c}_1^{(y)}t_2^{(y)} = 0 - (-1)\frac{5}{14} = \frac{5}{14}.\end{aligned}\tag{82}$$

Análogamente, para $i = 3$ y $i = 4$:

$$\begin{aligned}\mathbf{t}_{i=2}^{(y)} &= [5/14, 5/14, 5/14]^T, \\ \mathbf{t}_{i=3}^{(y)} &= [10/7, 10/7, 10/7]^T, \\ \mathbf{t}_{i=4}^{(y)} &= [75/14, 75/14, 75/14]^T.\end{aligned}\tag{83}$$

Al finalizar la primera iteración completa (barrido x y barrido y), el campo queda:

$$T^{(1)} = \begin{bmatrix} 0 & 5/14 & 10/7 & 75/14 & 0 \\ 0 & 5/14 & 10/7 & 75/14 & 10 \\ 0 & 5/14 & 10/7 & 75/14 & 0 \end{bmatrix},\tag{84}$$

Finalmente, el residuo que usa el programa es:

$$R^{(1)} = \left[\sum_{i=1}^5 \sum_{j=1}^3 \left(T_{i,j}^{(1)} - T_{i,j}^{(0)} \right)^2 \right]^{1/2} \approx 13.88,\tag{85}$$

valor muy superior a la tolerancia, por lo que el ciclo iterativo continúa.

A continuación se explica la implementación computacional del algoritmo secuencial para el caso 2D.

7 Metodología computacional secuencial del caso 2D

El algoritmo secuencial para el caso 2D se implementa en el programa `secuencial/calor2D/calor2D.f90` y `secuencial/calor2D/tridiagonal.f90`. Se resuelve iterativamente el sistema de ecuaciones generado por la discretización bidimensional de la ecuación de Laplace. Se usa el mismo algoritmo de Thomas para resolver los sistemas tridiagonales en cada dirección. El programa se ejecuta con `./calor2D` y los resultados se guardan en archivos de texto para su posterior análisis.

Se comienza con la implementación del programa principal `Calor2D.f90`, donde se declaran las variables y los parámetros necesarios para la simulación.

```
Program Calor2D
!
Implicit none
!
! Declaración de variables
!
! Iteradores y tamaño del problema
!
integer :: ii, jj, iter
integer, parameter :: nx = 60, ny = 30, itermax=10000
!
```

Seguidamente, se inicializan los parámetros del dominio computacional, incluyendo las dimensiones del dominio L_x y L_y , los pasos de malla Δx y Δy , y los inversos de los cuadrados de los pasos para optimizar los cálculos.

```
! Variables del dominio computacional
!
double precision :: lx, ly, deltax, deltay
double precision :: inv_dx2, inv_dy2
!
! Variable para el residuo de iteraciones, y tolerancia
!
double precision :: residuo, tolerancia, tolerancia2
```

Las variables del problema físico incluyen el vector de incógnitas tt para almacenar las temperaturas en cada iteración, los vectores tx y ty para almacenar las temperaturas en cada dirección, los vectores rx y ry para almacenar los resultados de los sistemas de ecuaciones, y los vectores cfx y cfy para almacenar las condiciones de frontera.

```
!
! Variables del problema físico
!
double precision :: tt(nx,ny,2)      ! vector de incógnitas, 1 para la iteración actual
                                      ! y 2 para la anterior
double precision :: tx(nx), ty(ny)   ! vectores de incógnitas 1D
double precision :: rx(nx), ry(ny)    ! vector de resultados del s. ecuaciones
double precision :: cfx(ny,2), cfy(nx,2) ! vector de condiciones de frontera
```

Asimismo, se necesita de una matriz tridiagonal sobredimensionada para cada dirección para almacenar los coeficientes de los sistemas tridiagonales. En este caso, se opta por usar vectores separados para cada dirección, lo que permite una implementación más clara y eficiente del algoritmo de Thomas para resolver los sistemas tridiagonales en cada dirección. Estos vectores se declaran para almacenar los coeficientes de los sistemas tridiagonales en cada dirección. Se declaran los vectores ax , bx_base y cx para la dirección x , y los vectores ay , by_base y cy para la dirección y . Además, se declaran los vectores cpx , $denx$, cpy y $deny$ para almacenar la factorización de Thomas.

```
!
! Variables para almacenar los coeficientes de los sistemas tridiagonales
!
double precision :: ax(nx), cx(nx), bx_base(nx) ! Variables para almacenar
                                                    ! matriz tridiagonal sobredimensionada
double precision :: cpx(nx), denx(nx)
!
```

```

double precision :: ay(ny), cy(ny), by_base(ny) ! Variables para almacenar
!
! matriz tridiagonal sobredimensionada
double precision :: cpy(ny), deny(ny)

```

A continuación, se establecen los parámetros de la simulación, incluyendo la tolerancia para la convergencia y las dimensiones del dominio computacional. Se calculan los pasos de malla y los inversos de los cuadrados de los pasos para optimizar los cálculos en el algoritmo de Thomas.

```

!
! Tolerancia
!
tolerancia = 1d-3
tolerancia2 = tolerancia*tolerancia
!
! Dominio computacional
!
lx      = 10.d0
deltax  = lx/(nx - 1) ! nx puntos definen (nx - 1) intervalos
ly      = 5.d0
deltay  = ly/(ny - 1) ! ny puntos definen (ny - 1) intervalos
inv_dx2 = 1.d0/(deltax*deltax)
inv_dy2 = 1.d0/(deltay*deltay)

```

Se realiza la inicialización de los vectores de coeficientes y del campo de temperaturas. Todos los vectores de coeficientes a_x , b_x , c_x para la dirección x y a_y , b_y , c_y para la dirección y , así como los vectores de términos independientes r_x y r_y .

Se crean los vectores ax , bx_base , cx , etc, que se irán llenando con los coeficiente de las matrices tridiagonales para cada dirección según las ecuaciones 54 - 56 y 60 - 62. El campo de temperatura $T(i, j, n)$ también se inicializa a cero, pudiendo modificarse posteriormente para establecer condiciones iniciales específicas:

```

!
! Inicializacion de variables
!
ax(:)    = 0.d0
bx_base(:) = 0.d0
cx(:)    = 0.d0
rx(:)    = 0.d0
ay(:)    = 0.d0
by_base(:) = 0.d0
cy(:)    = 0.d0
ry(:)    = 0.d0
tt(:, :, :) = 0.d0 ! Valores iniciales de temperatura, se pueden modificar
! para probar diferentes condiciones iniciales
!
! Coeficientes constantes de los sistemas tridiagonales.
!
do ii = 2, nx-1
    ax(ii) = inv_dx2
    bx_base(ii) = -2.d0*(inv_dx2 + inv_dy2)
    cx(ii) = inv_dx2
end do
ax(1) = 0.d0
bx_base(1) = 1.d0
cx(1) = 0.d0
ax(nx) = 0.d0
bx_base(nx) = 1.d0

```

```

cx(nx) = 0.d0
!
do jj = 2, ny-1
  ay(jj) = inv_dy2
  by_base(jj) = -2.d0*(inv_dx2 + inv_dy2)
  cy(jj) = inv_dy2
end do
ay(1) = 0.d0
by_base(1) = -1.d0
cy(1) = 1.d0
ay(ny) = -1.d0
by_base(ny) = 1.d0
cy(ny) = 0.d0

```

Ya que los coeficientes tridiagonales (a, b, c) son constantes durante todas las iteraciones, se pueden calcular una sola vez y almacenar en las operaciones de factorización con `tri_factor`.

```

!
call tri_factor(ax,bx_base,cx,cpx,dex,nx)
call tri_factor(ay,by_base,cy,cpy,deny,ny)

```

Asimismo, se está resolviendo una placa bidimensional, las condiciones frontera se expresarán como vectores para cada dirección. En este caso, se trabaja con condiciones mixtas: Dirichlet en dirección x y Neumann homogénea en dirección y . En x se fija la temperatura en los dos bordes ($T = 1^\circ C$ a la izquierda y $T = 0^\circ C$ a la derecha), mientras que en y se impone flujo nulo en ambos extremos ($\partial T / \partial y = 0$). Esto se traduce en

$$\begin{aligned}
cfx_1 = T_0 = 1, \quad cfy_1 = -k \frac{\partial T}{\partial y} \Big|_{y=0} &= 0, \\
cfx_2 = T_{L_x} = 0, \quad cfy_2 = -k \frac{\partial T}{\partial y} \Big|_{y=L_y} &= 0.
\end{aligned} \tag{86}$$

Donde, para la dirección x , se fija $T = 1^\circ C$ en el borde izquierdo y $T = 0^\circ C$ en el borde derecho.

Las 4 condiciones frontera se almacenan en los vectores cfx y cfy para cada dirección. Así,

$$\begin{aligned}
cfx(i,1) = 1, \quad cfx(i,2) = 0 \quad \forall i \in [1, n_x], \\
cfy(j,1) = 0, \quad cfy(j,2) = 0 \quad \forall j \in [1, n_y]
\end{aligned} \tag{87}$$

```

!
! Condiciones de frontera en direcci'on x
!
do ii = 1, ny
  cfx(ii,1) = 1.d0
  cfx(ii,2) = 0.d0
end do
!
! Condiciones de frontera en direcci'on y
!
do jj = 1, nx
  cfy(jj,1) = 0.d0
  cfy(jj,2) = 0.d0
end do

```

Luego, se ejecuta el bucle de iteraciones para resolver el sistema de ecuaciones. En cada iteración, se actualiza el valor de la iteración anterior, se ensamblan los sistemas tridiagonales para cada dirección y se resuelven usando el algoritmo de Thomas. El proceso se repite hasta alcanzar la convergencia o el número máximo de iteraciones.

Se comienza respaldando el valor de la iteración anterior en `tt(:, :, 2)` para poder calcular el residuo posteriormente. Luego, se realiza el barrido en la dirección y y dentro de este, el barrido en la dirección x para ensamblar los sistemas tridiagonales correspondientes a cada línea horizontal j .

```
bucle_iteraciones: do iter = 1, itermax
!
! Inicializamos el valor de la iteraci'on anterior
!
tt(:, :, 2) = tt(:, :, 1)
!
barrido_y: do jj = 2, ny-1
  ensambla_tri_x: do ii = 2, nx-1

    rx(ii) = -inv_dy2*tt(ii, jj-1, 1) - inv_dy2*tt(ii, jj+1, 1)

  end do ensambla_tri_x
```

El vector `rx` se llena según la ecuación 57 para cada nodo interior. Sus valores frontera se imponen según las condiciones de frontera almacenadas en `cfx`.

```
!
! Impone cond. frontera
!
rx(1)      = cfx(jj, 1)
!
rx(nx)     = cfx(jj, 2)
!
! Resolver el problema algebraico
!
call tri_solve(ax, cpx, denx, rx, tx, nx)
!
! Actualizar la temperatura de la placa
!
do ii = 1, nx

  tt(ii, jj, 1) = tx(ii)

end do
!
end do barrido_y
```

Luego se realiza el barrido complementario en la dirección x . En este caso, para cada columna interior i , se ensambla el sistema tridiagonal en y usando la ecuación 63. Después se imponen las fronteras con `cfy`, se resuelve el sistema y se actualiza la columna completa `tt(ii, jj, 1)`.

```
barrido_x: do ii = 2, nx-1
  ensambla_tri_y: do jj = 2, ny-1

    ry(jj) = -inv_dx2*tt(ii-1, jj, 1) - inv_dx2*tt(ii+1, jj, 1)

  end do ensambla_tri_y
!
! Impone cond. frontera
!
ry(1)      = cfy(ii, 1)
!
ry(ny)     = cfy(ii, 2)
```

```

!
! Resolver el problema algebraico
!
call tri_solve(ay,cpy,deny,ry,ty,ny)
!
! Actualizar la temperatura de la placa
!
do jj = 1, ny

    tt(ii,jj,1) = ty(jj)

end do
!
end do barrido_x

```

Con ambos barridos completados, el programa evalúa la convergencia mediante el residuo cuadrático entre la iteración actual y la anterior. El residuo se acumula sobre todos los nodos y se compara con `tolerancia2 = tolerancia*tolerancia`. Si el residuo es menor que ese umbral, se sale del bucle iterativo.

```

!
! Criterio de convergencia
!
residuo = 0.d0
do jj = 1, ny
    do ii = 1, nx
        residuo = residuo + (tt(ii,jj,1)-tt(ii,jj,2))*(tt(ii,jj,1)-tt(ii,jj,2))
    end do
end do
!
if( residuo < tolerancia2 )exit
!
end do bucle_iteraciones

```

Finalmente, al terminar el proceso iterativo, se reporta el número de iteraciones y se escribe el campo de temperatura en formato (x, y, T) , separando visualmente cada fila de y con una línea en blanco.

```

write(*,*) "Convergencia en ", iter, " iteraciones"
!
do jj = 1, ny
    do ii = 1, nx
        write(*,*) (ii-1)*deltax, (jj-1)*deltay, tt(ii,jj,1)
    end do
    write(*,*) ' '
end do
!
end Program Calor2D

```

8 Caso 2D paralelo con OpenMP

La versión paralela distribuye los barridos independientes entre hilos de OpenMP. Si t_p es el tiempo de ejecución usando p hilos y t_1 es el tiempo de referencia secuencial, el speedup se calcula como:

$$S_p = \frac{t_1}{t_p}. \quad (88)$$

La eficiencia paralela mide qué fracción del uso ideal de los p hilos se aprovecha:

$$E_p = \frac{S_p}{p}. \quad (89)$$

El caso ideal corresponde a:

$$S_p^{ideal} = p, \quad E_p^{ideal} = 1. \quad (90)$$

9 Resultados

Las tablas de rendimiento se guardan en `docs/tables/` y las figuras en `docs/figures/`. Para cada número de hilos p , el tiempo reportado se puede obtener como el promedio de varias corridas:

$$\bar{t}_p = \frac{1}{N} \sum_{\ell=1}^N t_{p,\ell}, \quad (91)$$

donde N es el número de repeticiones y $t_{p,\ell}$ es el tiempo medido en la corrida ℓ usando p hilos. Con este promedio, el speedup experimental queda:

$$S_p^{real} = \frac{\bar{t}_1}{\bar{t}_p}. \quad (92)$$

La eficiencia experimental se calcula como:

$$E_p^{real} = \frac{S_p^{real}}{p} = \frac{\bar{t}_1}{p \bar{t}_p}. \quad (93)$$

Para comparar con la aceleración ideal se usa:

$$\Delta S_p = S_p^{ideal} - S_p^{real} = p - S_p^{real}. \quad (94)$$

Una diferencia ΔS_p cercana a cero indica que la ejecución paralela se aproxima al comportamiento ideal. Cuando ΔS_p crece, la eficiencia baja por costos de sincronización, acceso a memoria, regiones no paralelas o desbalance entre hilos.

9.1 Comparación final con el código de referencia

Se comparó la versión paralela desarrollada en `paralelo/src/` contra el código de referencia proporcionado en `paralelo/reference/`. Los archivos de referencia no fueron modificados. Para reducir el efecto de arranques fríos y procesos de fondo, se usó una corrida de calentamiento por número de hilos y se reportó la mediana de seis corridas medidas.

Para aislar el efecto del paralelismo bajo el mismo comportamiento numérico, en la versión desarrollada se reprodujeron dos errores presentes en `reference/`: (i) la condición de frontera inferior del barrido en dirección y se dejó con la misma formulación incorrecta, y (ii) el corte por convergencia basado en residuo se mantuvo desactivado, forzando la ejecución hasta `itermax`.

La comparación debe leerse como una comparación de comportamiento paralelo en la misma máquina, no como igualdad estricta de carga numérica: ambas versiones usan una malla de 480×240 .

Cuadro 1: Comparación de tiempos y aceleración en la misma máquina.

Hilos	t_{nuestra} [s]	S_{nuestra}	t_{ref} [s]	S_{ref}
1	40.466	1.000	36.954	1.000
2	21.829	1.854	21.855	1.691
3	15.465	2.617	15.736	2.348
4	13.534	2.990	14.136	2.614
5	12.845	3.150	10.911	3.387
6	11.342	3.568	9.832	3.759
7	8.554	4.731	8.890	4.157
8	8.102	4.994	10.386	3.558

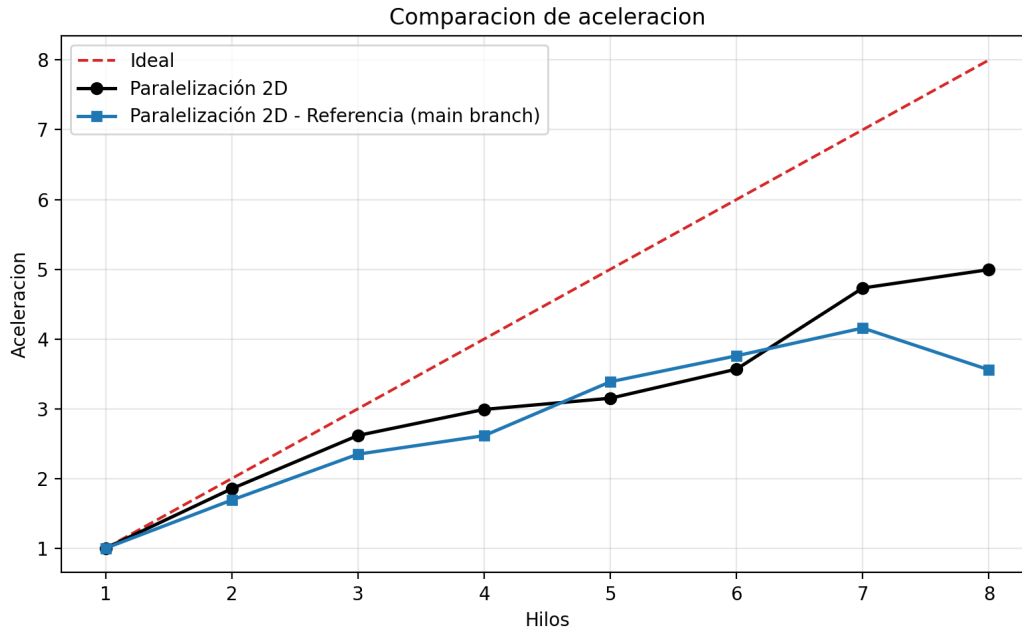


Figura 3: Curvas de aceleración de la versión desarrollada y la referencia.

10 Conclusiones

Referencias